

그리디(Greedy) 알고리즘

동양미래대학교

컴퓨터소프트웨어공학과

송원문

mooniesong@gmail.com

목차

- 그리디 알고리즘
- 예제 – 거스름 돈
- 실습 – 1이 될 때까지

- 문제 – 모험가 길드
- 문제 – 곱하기 혹은 더하기

그리디 알고리즘

■ 그리디 알고리즘 (Greedy, 탐욕법)

- 현재 상황에서 지금 당장 좋은 것만 고르는 방법
- 현재의 상황이 나중에 미칠 영향에 대해서는 고려하지 않음

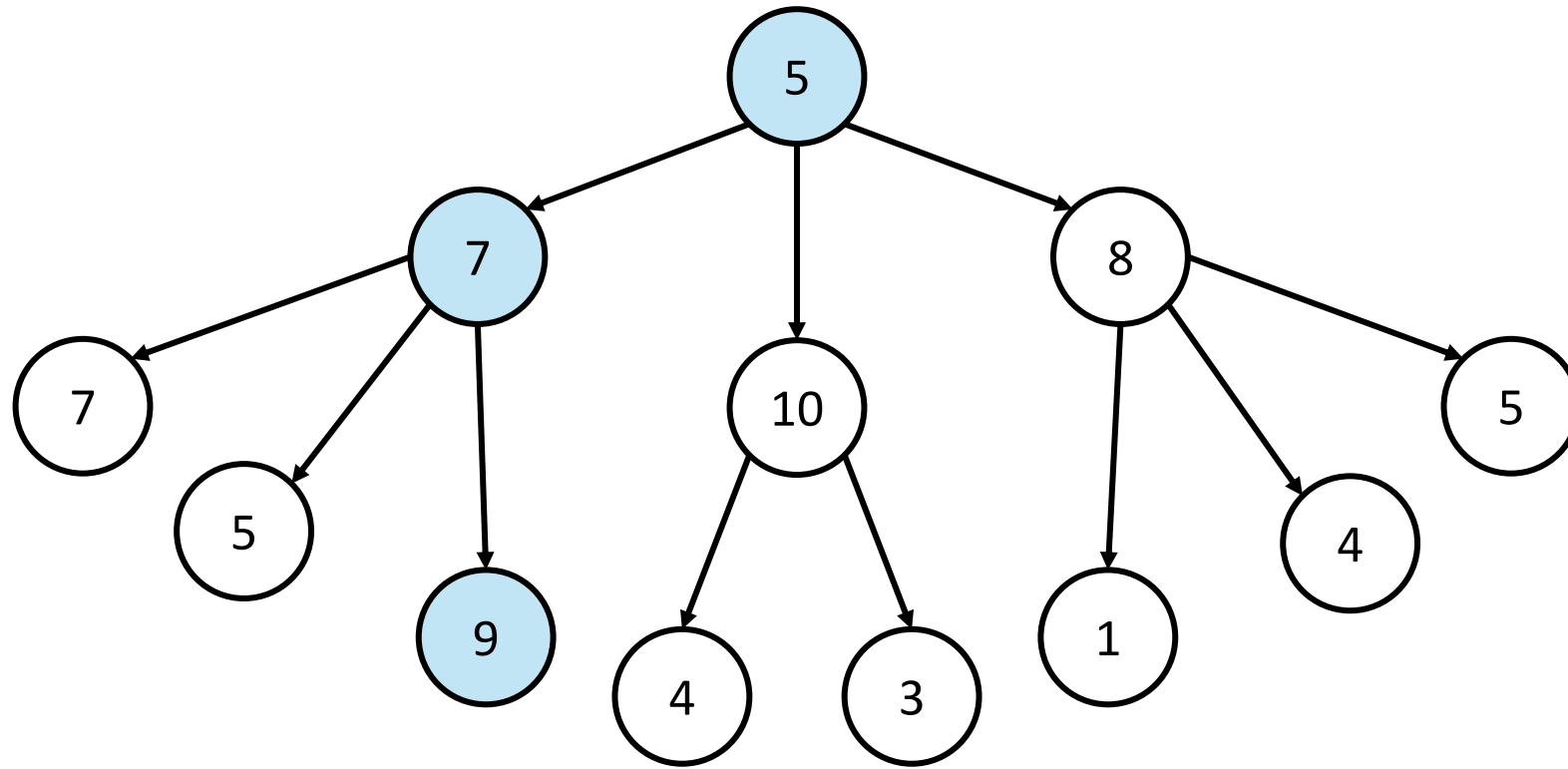
■ 일반적인 그리디 알고리즘은 문제를 풀기 위한 최소한의 아이디어를 떠올릴 수 있는 능력을 요구

■ 그리디 해법은 그 정당성 분석이 중요

■ 단순히 가장 좋아 보이는 것을 반복적으로 선택해도 최적의 해를 구할 수 있는지 검토 필요

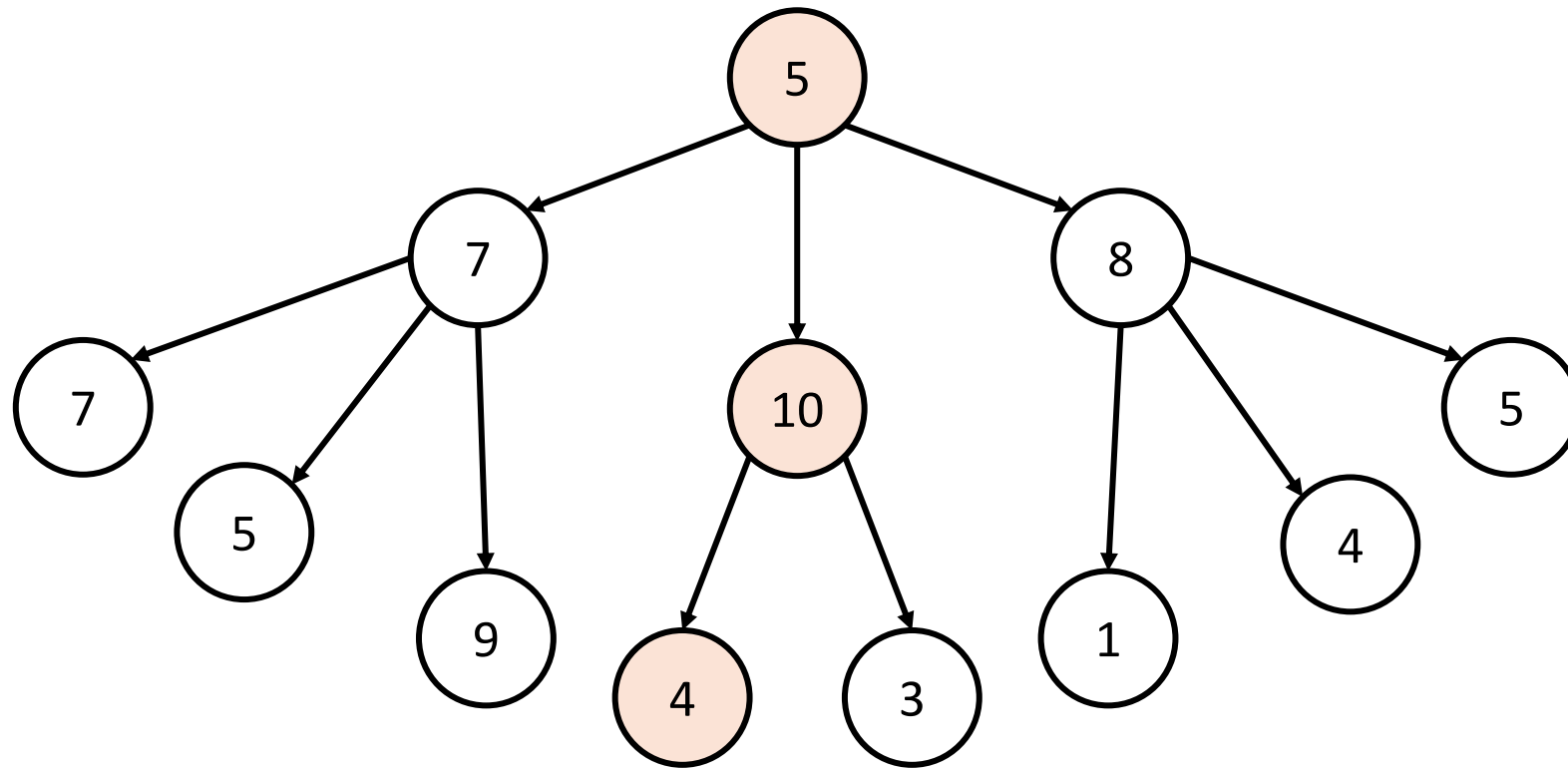
■ [문제 상황] 루트 노드부터 시작하여 거쳐 가는 노드 값의 합을 최대로 만들기

- Q. 최적의 해는?



■ [문제 상황] 루트 노드부터 시작하여 거쳐 가는 노드 값의 합을 최대로 만들기

- Q. 단순히 매 상황에서 가장 큰 값만 고른다면? – 그리디 방법에서의 탐색



그리디 알고리즘

- 일반적인 상황에서 그리디 알고리즘은 최적의 해를 보장할 수 없을 때가 많음
- 하지만 코딩 테스트에서의 대부분의 그리디 문제는 탐욕법으로 얻은 해가 최적의 해가 되는 상황에서, 이를 추론할 수 있어야 풀리도록 출제

예제 – 거스름 돈

■ 문제 설명

- 당신은 음식점의 계산을 도와주는 점원
- 카운터에는 거스름돈으로 사용할 500원, 100원, 50원, 10원짜리 동전이 무한히 존재한다고 가정
- 손님에게 거슬러 주어야 할 돈이 N원일 때 거슬러 주어야 할 동전의 최소 개수를 구하라
- 단, 거슬러 줘야 할 돈 N은 항상 10의 배수



예제 – 거스름 돈

■ 문제 해결 아이디어

- 최적의 해를 빠르게 구하기 위해서는 가장 큰 화폐 단위부터 돈을 거슬러 주면 됨
- N원을 거슬러 줘야 할 때, 가장 먼저 500원으로 거슬러 줄 수 있을 만큼 거슬러 줌
이후에 100원, 50원, 10원짜리 동전을 차례대로 거슬러 줄 수 있을 만큼 거슬러 주면 됨

■ $N = 1,260$ 일 때

예제 – 거스름 돈

■ N = 1,260일 때

- [Step 0] 초기 단계 – 남은 돈: 1,260원

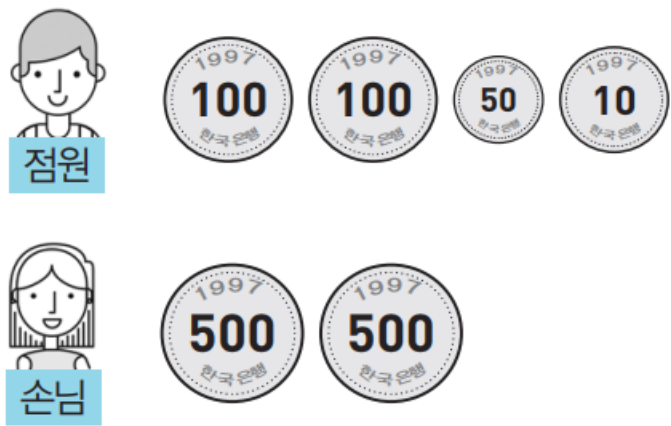


본래 점원이 거슬러 줘야 할 돈은 보이지 않고
잔돈 무더기가 있을 것.
예제에서는 이해를 돕고자 점원이 줘야 하는
거스름돈에 대해서 시각적으로 표현

화폐 단위	500	100	50	10
손님이 받은 개수	0	0	0	0

예제 – 거스름 돈

- N = 1,260일 때
 - [Step 1] 남은 돈: 260원



화폐 단위	500	100	50	10
손님이 받은 개수	2	0	0	0

예제 – 거스름 돈

- N = 1,260일 때
 - [Step 2] 남은 돈: 60원



화폐 단위	500	100	50	10
손님이 받은 개수	2	2	0	0

예제 – 거스름 돈

■ N = 1,260일 때

- [Step 3] 남은 돈: 10원



화폐 단위	500	100	50	10
손님이 받은 개수	2	2	1	0

예제 – 거스름 돈

- N = 1,260일 때
 - [Step 4] 남은 돈: 0원



(아무것도 없는 상태)



화폐 단위	500	100	50	10
손님이 받은 개수	2	2	1	1

예제 – 거스름 돈

■ 정당성 분석

- 가장 큰 화폐 단위부터 돈을 거슬러 주는 것이 최적의 해를 보장하는 이유는?
 - 가지고 있는 동전 중에서 큰 단위가 항상 작은 단위의 배수이므로 작은 단위의 동전들을 종합해 큰 단위 보다 더 좋은(작은) 다른 해가 나올 수 없기 때문
- 만약에 800원을 거슬러 주어야 하는데 화폐 단위가 500원, 400원, 100원이라면?
 - 큰 단위가 항상 작은 단위의 배수라는 앞서 언급한 기존 가정이 깨짐. (500원은 400원의 배수가 아님)
 - 따라서, 이 경우에는 작은 단위의 동전들을 종합해 큰 단위 보다 더 좋은(작은) 다른 해가 나올 수 있음
- 그리디 알고리즘 문제에서는 이처럼 문제 풀이를 위한 최소한의 아이디어를 떠올리고 이것이 정당한지 검토할 수 있어야 함

예제 – 거스름 돈

■ 파이썬 코드 예시

```
n = 1260
count = 0

# 큰 단위의 화폐부터 차례대로 확인하기
array = [500, 100, 50, 10]

for coin in array:
    count += n // coin # 해당 화폐로 거슬러 줄 수 있는 동전의 개수 세기
    n %= coin

print(count)
```

■ 코드의 시간 복잡도

- 화폐의 종류가 K 개라고 할 때, 소스코드의 시간 복잡도는 $O(K)$
- 이 알고리즘의 시간 복잡도는 거슬러줘야 하는 금액(변수의 크기)과는 무관하며, 동전의 총 종류에만 영향을 받음

실습 – 1이 될 때까지

■ 문제 설명

- 어떠한 수 N 이 1이 될 때까지 다음의 두 과정 중 하나를 반복적으로 선택하여 수행하려고 함. 단, 두 번째 연산은 N 이 K 로 나누어 떨어질 때만 선택할 수 있음
 - N 에서 1을 뺌. or N 을 K 로 나눈다.
- 예를 들어 N 이 17, K 가 4라고 가정
 - 이때 1번의 과정을 한 번 수행하면 N 은 16이 됨. 이후에 2번의 과정을 두 번 수행하면 N 은 1이 됨
 - 결과적으로 이 경우 전체 과정을 실행한 횟수는 3이 됨. 이는 N 을 1로 만드는 최소 횟수.
- N 과 K 가 주어질 때 N 이 1이 될 때까지 1번 혹은 2번의 과정을 수행해야 하는 최소 횟수를 구하는 프로그램을 작성하라

실습 – 1이 될 때까지

■ 문제 조건

난이도 ●○○ | 풀이 시간 15분 | 시간제한 2초 | 메모리 제한 128MB

입력 조건

- 첫째 줄에 $N(1 \leq N \leq 100,000)$ 과 $K(2 \leq K \leq 100,000)$ 가 공백을 기준으로 하여 각각 자연수로 주어집니다.

출력 조건

- 첫째 줄에 N 이 1이 될 때까지 1번 혹은 2번의 과정을 수행해야 하는 횟수의 최솟값을 출력합니다.

입력 예시

25 5

출력 예시

2

실습 – 1이 될 때까지

■ 문제 해결 아이디어

- 주어진 N 에 대하여 최대한 많이 나누기를 수행하면 됨
- N 의 값을 줄일 때 2 이상의 수로 나누는 작업이 1을 빼는 작업보다 수를 훨씬 많이 줄일 수 있음
- 예를 들어 $N = 25$, $K = 3$ 일 때는 다음과 같음

단계	연산 과정	N 의 값
0단계(초기 단계)		$N = 25$
1단계	N 에서 1 빼기	$N = 24$
2단계	N 을 K 로 나누기	$N = 8$
3단계	N 에서 1 빼기	$N = 7$
4단계	N 에서 1 빼기	$N = 6$
5단계	N 을 K 로 나누기	$N = 2$
6단계	N 에서 1 빼기	$N = 1$

실습 – 1이 될 때까지

■ 정당성 분석

- 가능하면 최대한 많이 나누는 작업이 최적의 해를 항상 보장할 수 있을까?
- N 이 아무리 큰 수여도, K 로 계속 나눈다면 기하급수적으로 빠르게 줄일 수 있음
- 다시 말해, K 가 2 이상이기만 하면, K 로 나누는 것이 1을 빼는 것보다 항상 빠르게 N 을 줄일 수 있는 것
- 또한 N 은 항상 1에 도달하게 됨 (최적의 해 성립)

실습 – 1이 될 때까지

■ 파이썬 코드 예시

```
# N, K을 공백을 기준으로 구분하여 입력 받기
n, k = map(int, input().split())

result = 0

while n >= k:
    # N이 K로 나누어 떨어지지 않는다면 N에서 1씩 빼기
    while n % k != 0:
        n -= 1
        result += 1

    # K로 나누기
    n //= k
    result += 1

# 마지막으로 남은 수에 대하여 1씩 빼기
while n > 1:
    n -= 1
    result += 1

print(result)
```

실습 – 1이 될 때까지

■ 파이썬 코드 예시

- N이 아주 큰 숫자라면? K로 나누어 떨어질 때까지 계속 1을 빼는 것 (횟수를 1씩 늘리는 것)이 비 효율적일 수 있음
- 다시 말하면, N이 K의 배수가 되도록 설정하고 그 값과의 차이를 한번에 빼는 방식 (횟수를 한번에 늘리는 방식)으로 수정하면, 실행 횟수가 더 줄어들 수 있음

실습 – 1이 될 때까지

■ 파이썬 코드 예시

```
# N, K을 공백을 기준으로 구분하여 입력 받기
n, k = map(int, input().split())

result = 0

while n >= k:
    # N이 K로 나누어 떨어지는 수가 될 때까지 빼기
    target = (n // k) * k
    result += (n - target)
    n = target
    # N이 K보다 작을 때 (더 이상 나눌 수 없을 때) 반복문 탈출
    if n < k:
        break
    # K로 나누기
    result += 1
    n //= k

# 마지막으로 남은 수에 대하여 1씩 빼기
result += (n - 1)

print(result)
```

실습 – 1이 될 때까지

■ 파이썬 코드 예시

```
# N, K을 공백을 기준으로 구분하여 입력 받기
n, k = map(int, input().split())

result = 0

while n >= k:
    # N이 K로 나누어 떨어지는 수가 될 때까지 빼기
    target = (n // k) * k
    result += (n - target)
    n = target
    # N이 K보다 작을 때 (더 이상 나눌 수 없을 때) 반복문 탈출
    if n < k:
        break
    # K로 나누기
    result += 1
    n //= k

# 마지막으로 남은 수에 대하여 1씩 빼기
result += (n - 1)

print(result)
```

[질문] if $n < k$ 인 경우가 있을 수 있는가?
없거나 있는경우에, 더 효율적인 알고리즘은 있는가?

목차

- 그리디 알고리즘
- 예제 – 거스름 돈
- 실습 – 1이 될 때까지

- 문제 – 모험가 길드
- 문제 – 곱하기 혹은 더하기

문제 – 모험가 길드

■ 문제 설명

- 한 마을에 모험가가 N명 있음. 모험가 길드에서는 N명의 모험가를 대상으로 ‘공포도’를 측정했는데, ‘공포도’가 높은 모험가는 쉽게 공포를 느껴 위험 상황에서 제대로 대처할 능력이 떨어짐
- 모험가 길드장인 동빈이는 모험가 그룹을 안전하게 구성하고자 공포도가 X인 모험가는 반드시 X명 이상으로 구성한 모험가 그룹에 참여해야 여행을 떠날 수 있도록 규정을 만들
- 동빈이는 최대 몇 개의 모험가 그룹을 만들 수 있는가? N명의 모험가에 대한 정보가 주어졌을 때, 여행을 떠날 수 있는 그룹 수의 최댓값을 구하는 프로그램을 작성하라

- 예를 들어 $N = 5$ 이고, 각 모험가의 공포도가 다음과 같다고 가정하면,

2 3 1 2 2

- 이 경우, 그룹 1에 공포도가 1, 2, 3인 모험가를 한 명씩 넣고, 그룹 2에 공포도가 2인 남은 두 명을 넣게 되면 총 2개의 그룹을 만들 수 있음
- 또한 몇 명의 모험가는 마을에 그대로 남아 있어도 되기 때문에, 모든 모험가를 특정한 그룹에 넣을 필요는 없음

문제 – 모험가 길드

■ 문제 조건

난이도 ●○○ | 풀이 시간 30분 | 시간 제한 1초 | 메모리 제한 128MB | 기출 핵심 유형

입력 조건

- 첫째 줄에 모험가의 수 N 이 주어집니다. ($1 \leq N \leq 100,000$)
- 둘째 줄에 각 모험가의 공포도의 값을 N 이하의 자연수로 주어지며, 각 자연수는 공백으로 구분합니다.

출력 조건

- 여행을 떠날 수 있는 그룹 수의 최댓값을 출력합니다.

입력 예시

```
5
2 3 1 2 2
```

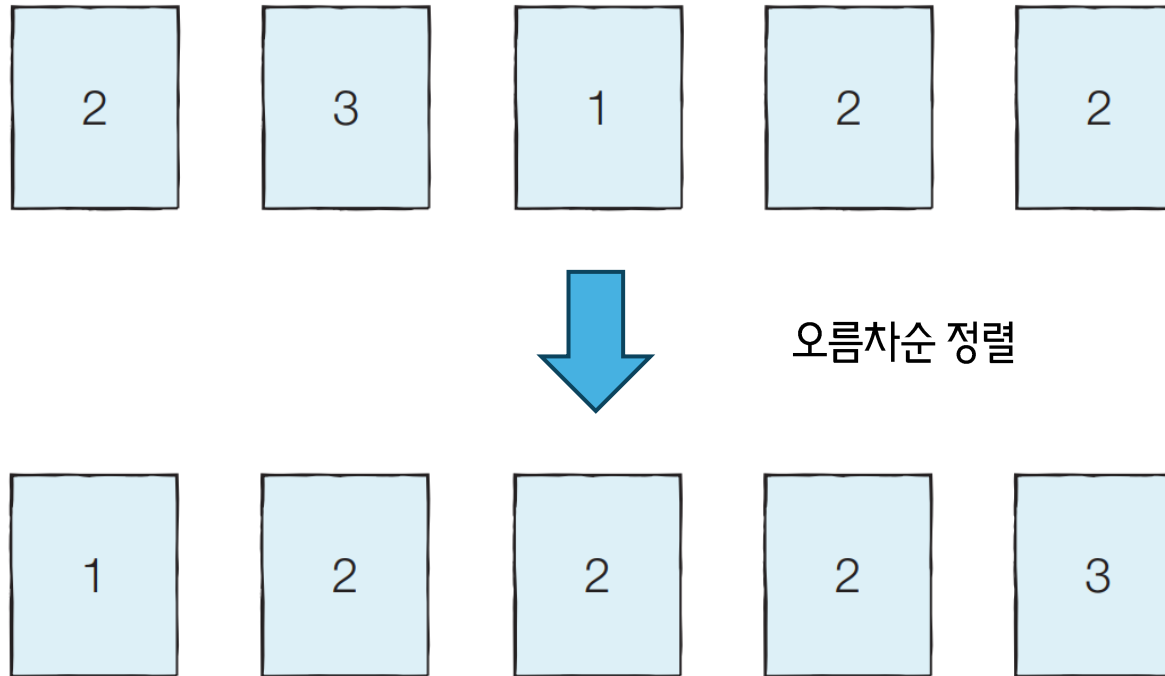
출력 예시

```
2
```

문제 – 모험가 길드

■ 문제 해결 아이디어

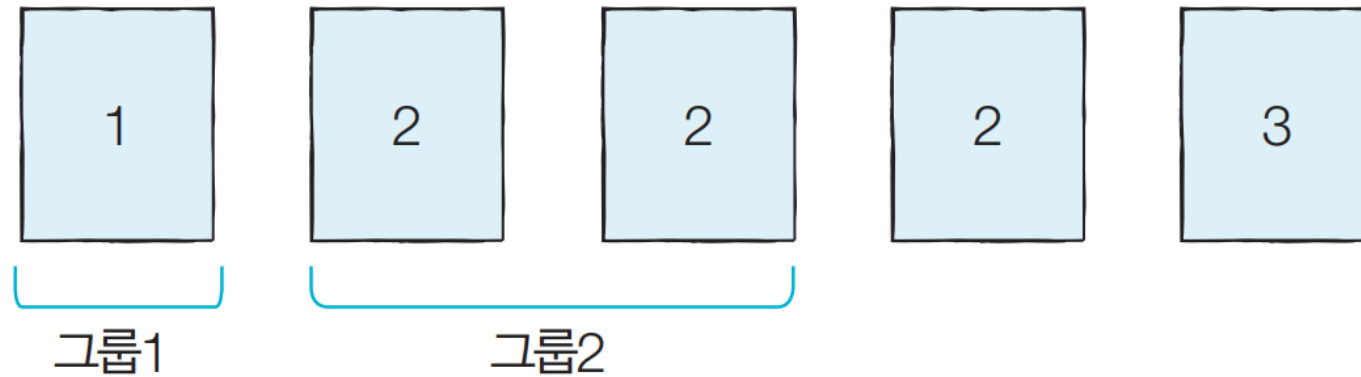
- 오름차순 정렬 이후에 공포도가 가장 낮은 모험가부터 하나씩 확인



문제 – 모험가 길드

■ 문제 해결 아이디어

- 앞에서부터 공포도를 하나씩 확인하며 ‘현재 그룹에 포함된 모험가의 수’가 ‘현재 확인하고 있는 공포도’보다 크거나 같다면 이를 그룹으로 설정



- 공포도가 오름차순으로 정렬되어 있기 때문에, 항상 최소한의 모험가의 수만 포함하여 그룹을 결성하게 됨
최소한의 모험가의 수만 포함하여 그룹을 결성하면, 최대한의 그룹 수를 만들 수 있음

문제 – 모험가 길드

■ 파이썬 코드 예시

```
n = int(input())
data = list(map(int, input().split()))
data.sort()

result = 0 # 총 그룹의 수
count = 0 # 현재 그룹에 포함된 모험가의 수

for i in data: # 공포도를 낮은 것부터 하나씩 확인하며
    count += 1 # 현재 그룹에 해당 모험가를 포함시키기
    if count >= i: # 현재 그룹에 포함된 모험가의 수가 현재의 공포도 이상이라면, 그룹 결성
        result += 1 # 총 그룹의 수 증가시키기
        count = 0 # 현재 그룹에 포함된 모험가의 수 초기화

print(result) # 총 그룹의 수 출력
```

문제 – 곱하기 혹은 더하기

■ 문제 설명

- 각 자리가 숫자(0부터 9)로만 이루어진 문자열 S 가 주어졌을 때, 왼쪽부터 오른쪽으로 하나씩 모든 숫자를 확인하며 숫자 사이에 '×' 혹은 '+' 연산자를 넣어 결과적으로 만들어질 수 있는 가장 큰 수를 구하는 프로그램을 작성하라
- 단, +보다 ×를 먼저 계산하는 일반적인 방식과는 달리, 모든 연산은 왼쪽에서부터 순서대로 이루어진다고 가정
- 예를 들어 02984라는 문자열로 만들 수 있는 가장 큰 수는 $((((0 + 2) \times 9) \times 8) \times 4) = 576$
- 또한 만들어질 수 있는 가장 큰 수는 항상 20억 이하의 정수가 되도록 입력이 주어짐

문제 – 곱하기 혹은 더하기

■ 문제 조건

난이도 ●○○ | 풀이 시간 30분 | 시간 제한 1초 | 메모리 제한 128MB | 기출 Facebook 인터뷰

입력 조건

- 첫째 줄에 여러 개의 숫자로 구성된 하나의 문자열 S가 주어집니다. ($1 \leq S$ 의 길이 ≤ 20)

출력 조건

- 첫째 줄에 만들어질 수 있는 가장 큰 수를 출력합니다.

입력 예시 1

02984

출력 예시 1

576

입력 예시 2

567

출력 예시 2

210

문제 – 곱하기 혹은 더하기

■ 문제 해결 아이디어

- 대부분의 경우 ‘+’보다는 ‘×’가 더 값을 크게 만들
- 예를 들어 $5 + 6 = 11$ 이고, $5 \times 6 = 30$
- 다만 두 수 중에서 하나라도 ‘0’ 혹은 ‘1’인 경우, 곱하기보다는 더하기를 수행하는 것이 필요
- 따라서 두 수에 대하여 연산을 수행할 때, 두 수 중에서 하나라도 1 이하인 경우에는 더하며, 두 수가 모두 2 이상인 경우에는 곱하는 것이 정답

문제 – 곱하기 혹은 더하기

■ 파이썬 코드 예시

```
data = input()

# 첫 번째 문자를 숫자로 변경하여 대입
result = int(data[0])

for i in range(1, len(data)):
    # 두 수 중에서 하나라도 '0' 혹은 '1'인 경우, 곱하기보다는 더하기 수행
    num = int(data[i])
    if num <= 1 or result <= 1:
        result += num
    else:
        result *= num

print(result)
```